

4. Banco de Preguntas Diagnósticas

Se presentan **6 bancos de 15 preguntas** cada uno, uno por categoría. Cada alternativa incorrecta está mapeada explícitamente al error que diagnostica, junto con el razonamiento erróneo que llevó al estudiante a elegirla.

4.1. Banco C — Errores Conceptuales y de Tipos de Datos

C1. ¿Cuál es el resultado de ejecutar `c = 2b + 3` cuando `b = 4`?

- **A) Produce un error de sintaxis.** *(Correcta — `2b` no es un token válido en Python, en cambio, `2 * b` sí lo es; el código falla antes de ejecutarse.)*
- B) Imprime 11. [C1]
- C) Imprime 24. [C1]
- D) Imprime 19. [C1]

C2. ¿Qué imprime `print("3" + "5")`?

- A) 8 [C2]
- **B) "35"** *(Correcta — Al ser ambos elementos de tipo `string`, Python los concatena.)*
- C) "8" [C2]
- D) Error de tipo [C2]

C3. ¿Qué resultado produce `x = int("7") + 3`?

- A) "73" [C2]
- B) Error de tipo [C2]
- **C) 10** *(Correcta — `int("7")` convierte el `string` a entero y la suma es aritmética.)*
- D) Python despeja `x` como si fuera una ecuación y obtiene 10 directamente sin necesitar el `int()`. [C1]

C4. ¿Qué pasa al ejecutar `y = x + 2 = 10`?

- **A) Error de sintaxis.** *(Correcta — no se puede tener dos `=` en una misma expresión de asignación simple.)*
- B) `x` queda en 8 e `y` en 10. [C1]
- C) `y` queda en 10 y `x` queda indefinida. [C1]
- D) Python resuelve que `x = 8`. [C1].

C5. ¿Cuál es el tipo de dato de la variable `resultado` al ejecutar: `resultado = "10" * 2`?

- A) `int`, vale 20. [C2]
- B) Error, no se puede multiplicar un `string`. [C2]
- **C) `str`, vale "1010".** *(Correcta — el operador `*` entre un `string` y un entero repite el `string`.)*
- D) `float`, vale 20.0. [C2]

C6. ¿Cuál de las siguientes expresiones produce `True`?

- A) `"5" == 5` [C2]
- B) `5.0 == "5.0"` [C2]
- **C) `5.0 == 5`** *(Correcta — Python permite comparar `int` y `float` numéricamente.)*
- D) `5 == 2 + 3b` produce `True` cuando `b = 1` [C1]

C7. ¿Qué imprime `print("hola" * 0)`?

- A) (cadena vacía) *(Correcta — multiplicar un string por 0 produce el string vacío "").*
- B) "hola0" [C1]
- C) Error de tipo [C2]
- D) 0 [C2]

C8. ¿Qué pasa al ejecutar `x = 3a`?

- A) **Error de sintaxis.** *(Correcta — 3a no es un identificador válido ni un número válido en Python.)*
- B) Python interpreta `3a` como `3 * a`. [C1]
- C) `x` vale 0, porque Python interpreta `3a` como `3 * a` y al no estar definida `a` asume que vale 0. [C1]
- D) Depende del valor de `a`. [C1]

C9. ¿Qué produce `print(len("123"))`?

- A) 123 [C2]
- B) 3.0 [C2]
- C) **3** *(Correcta — len cuenta la cantidad de caracteres, no el valor numérico.)*
- D) Python calcula el valor del string como número y devuelve 123. [C1]

C10. ¿Qué imprime el siguiente código?

```
a = "5"
b = 2
print(a * b)
```

- A) 10 [C2]
- B) Error de tipo [C2]
- C) "55" *(Correcta — el operador * repite el string "5" dos veces.)*
- D) "5 * 2" [C2]

C11. ¿Qué pasa si se ejecuta `n = 5 + "5"`?

- A) `n` vale 10. [C2]
- B) `n` vale "55". [C2]
- C) **TypeError.** *(Correcta — Python no convierte implícitamente entre int y str para el operador +.)*
- D) `n` vale "105". [C2]

C12. ¿Qué imprime el siguiente código?

```
x = "4"
y = "2"
print(x - y)
```

- A) 2 [C2]
- B) "2" [C2]
- C) "42" [C2]
- D) **TypeError.** *(Correcta — el operador - no está definido para strings; a diferencia de +, Python no tiene una interpretación textual para la resta.)*

C13. ¿Qué ocurre al ejecutar el siguiente código?

```
b = 5
c = 3
print(bc)
```

- A) Imprime 53. [C1]
- B) Imprime 15. [C1]
- C) `NameError`. *(Correcta — `bc` no es una multiplicación implícita; Python lo interpreta como un identificador no definido y lanza `NameError`.)*
- D) Imprime "53". [C2]

C14. Un estudiante escribe `resultado = 2k - 1` siendo `k = 4`. ¿Qué ocurre?

- A) `SyntaxError`. *(Correcta — `2k` no es válido en Python; la multiplicación implícita no existe en el lenguaje.)*
- B) `resultado` vale 7. [C1]
- C) `resultado` vale "`2k - 1`". [C2]
- D) Python interpreta `2k` como `str(2) + str(k)` y produce "24". [C2]

C15. ¿Qué imprime el siguiente código?

```
edad = input("Ingresa tu edad: ")
nueva_edad = edad + 1
print(nueva_edad)
```

(Considera que el usuario ingresa 20.)

- A) 21 [C2]
- B) "201" [C2]
- C) 201 [C2]
- D) `TypeError`. *(Correcta — `input()` siempre retorna un `str`; sumar un `string` con un entero sin conversión explícita lanza `TypeError`.)*

4.2. Banco V — Errores de Asignación de Variables

V1. ¿Cuál de las siguientes instrucciones es sintácticamente correcta?

- A) `x = 10` *(Correcta — el identificador está a la izquierda y el valor a la derecha.)*
- B) `10 = x` [V1]
- C) `x + 1 = 10` [V1]
- D) `x == 10` [V4]

V2. ¿Qué imprime el siguiente código?

```
x = 10
y = x
x = 20
print(y)
```

- A) 10 *(Correcta — `y` copió el valor de `x` en ese instante; reasignar `x` no afecta a `y`.)*
- B) 20 [V2]
- C) Error de sintaxis: la asignación `y = x` es inválida; debería escribirse `x = y`. [V1]
- D) Error [V2]

V3. ¿Qué imprime este código?

```
a = 5
a = a + 3
print(a)
```

- A) 5 [S1]
- B) 8 (Correcta — el lado derecho se evalúa primero con el valor actual de **a** (5), luego se asigna el resultado (8).)
- C) Error: no se puede usar **a** en su propia asignación. [V1]
- D) **a + 3** [V1]

V4. ¿Qué imprime este código si **z no fue declarada antes?**

```
print(z)
```

- A) **NameError** (Correcta — en Python, intentar leer una variable no definida lanza **NameError**.)
- B) Imprime 0. [V3]
- C) Imprime None. [V3]
- D) Imprime "". [V3]

V5. ¿Qué imprime este código?

```
b = (3 > 1)
print(b)
```

- A) Error de sintaxis en la línea 1. [V4]
- B) **True** (Correcta — los operadores relacionales o de comparación, tales como: **>**, **<**, **>=**, **<=**, **=**, **!=**, siempre retornan un booleano (**True** or **False**))
- C) 3 [V4]
- D) "3 >1" [V4]

V6. ¿Qué valor tiene **x al final del programa?**

```
x = 1
y = x
y = 99
print(x)
```

- A) **1** (Correcta — **y = x** copió el valor 1; reasignar **y** no altera **x**.)
- B) 99 [V2]
- C) None [V2]
- D) Error [V2]

V7. ¿Qué pasa si se ejecuta **5 = n?**

- A) **SyntaxError** (Correcta — un literal numérico no puede estar a la izquierda del operador de asignación.)
- B) **n** pasa a valer 5. [V1]
- C) Se asigna la dirección de memoria de **n** al literal 5. [V1]
- D) Depende de si **n** fue definida antes. [V3]

V8. ¿Qué imprime el siguiente código?

```
flag = (10 == 10)
if flag:
    print("igual")
```

- A) No imprime nada porque `flag` no es un `if`. [V4/]
- B) Error: no se puede usar una variable como condición. [V4/]
- C) igual (Correcta — `flag` contiene `True`, que es un valor booleano válido para un `if`.)
- D) Imprime `True`. [V4/]

V9. ¿Qué ocurre si se ejecuta `total += 1` sin haber declarado `total`?

- A) `total` pasa a valer 1. [V3/]
- B) `total` pasa a valer 0. [V3/]
- C) `total` queda como `None`. [V3/]
- D) `NameError`. (Correcta — `+=` necesita leer `total` antes de actualizarlo, y si no existe, lanza `NameError`.)

V10. ¿Es válido este código? `if (a >b) == True:`

- A) No, `a >b` no puede compararse con `True`. [V4/]
- B) Sí, y es la única forma correcta de escribir una condición. [V4/]
- C) Sí, pero es redundante: `if a >b:` es equivalente y preferible. (Correcta — `a >b` ya produce un booleano; compararlo con `True` es innecesario.)
- D) No, los paréntesis en un `if` son siempre un error sintáctico. [V1/]

V11. ¿Qué imprime este código?

```
x = 5
x = x * 2
x = x - 3
print(x)
```

- A) 5 [V2/]
- B) 7 (Correcta — $5 \cdot 2 = 10$, luego $10 - 3 = 7$; cada asignación sobrescribe el valor anterior.)
- C) `x * 2 - 3` [V1/]
- D) 10 [V2/]

V12. ¿Qué imprime este código?

```
a = 1
b = 2
a = b
b = a
print(a, b)
```

- A) 1 2 [S1].
- B) 2 1 [V2/]
- C) 2 2 (Correcta — tras `a = b`, `a` vale 2; tras `b = a`, `b` también vale 2.)
- D) Error [V1/]

V13. ¿Qué imprime el siguiente código?

```
resultado = (5 < 3)
print(resultado)
```

- A) Error: no se puede asignar una comparación. [V4/]
- B) 3 [V4/]
- C) 0 [V4/]

- **D) False** *(Correcta — `5 < 3` es una expresión booleana que produce `False`, valor perfectamente asignable a una variable.)*

V14. ¿Qué valor tiene `p` tras ejecutar `p = p` si `p` ya estaba definida?

- A) 0 [V3]
- B) None [V3]
- **C) El mismo que tenía antes.** *(Correcta — Python evalúa el lado derecho primero; si `p` existía, su valor no cambia.)*
- D) None, porque `p = p` reemplaza el valor de `p` por su valor “por defecto”, como si la variable se reiniciara. [V3]

V15. ¿Qué imprime este código?

```
x = 10
y = x + 0
x = 99
print(y)
```

- A) 99 [V2]
- **B) 10** *(Correcta — `y` recibió el valor 10 en el momento de la asignación; reasignar `x` no la afecta.)*
- C) None [V3]
- D) `x + 0` [V1]

4.3. Banco S — Errores de Secuencias de Control

S1. ¿Qué imprime este código?

```
u = 0
if u < 5:
    print("menor")
    u = 10
```

- **A) Imprime menor una vez y termina.** *(Correcta — el `if` evalúa la condición una sola vez; el cambio de `u` dentro del bloque no reevalúa nada.)*
- B) Imprime menor indefinidamente. [S3]
- C) No imprime nada porque `u` cambia a 10. [S1]
- D) Imprime menor hasta que `u` vale 5. [S3, S4].

S2. ¿Qué produce este código?

```
i = 0
while i < 3:
    print(i)
    i += 1
```

- A) Imprime 1, 2, 3. [S1]
- **B) Imprime 0, 1, 2.** *(Correcta — `i` comienza en 0; la condición `i < 3` excluye el valor 3.)*
- C) Imprime 0, 1, 2, 3. [S4]
- D) Ciclo infinito. [S4]

S3. ¿Cuántas veces se ejecuta el bloque en el siguiente código?

```
x = 10
if x > 5:
    print("mayor")
else:
    print("menor o igual")
```

- A) Dos veces, una por cada rama. [S2]
- **B) Una vez, solo la rama if.** *(Correcta — if y else son mutuamente excluyentes; solo se ejecuta la rama cuya condición se cumple.)*
- C) Cero veces, el if y el else se cancelan. [S2]
- D) Depende del orden de evaluación del sistema. [S1]

S4. ¿Qué valor tiene b al finalizar?

```
a = 1
b = 0
if a > 0:
    b = 5
a = -1
print(b)
```

- A) 0, porque a cambia a -1 y el if se reevalúa retroactivamente. [S1]
- **B) 5** *(Correcta — el if se evaluó cuando a era 1; el cambio posterior de a no afecta a b.)*
- C) -1, porque b hereda el último valor de a. [S1]
- D) 0, porque el if repite su evaluación cuando a cambia a -1. [S3]

S5. ¿Qué imprime este código?

```
x = 5
if x > 0:
    if x > 10:
        print("grande")
else:
    print("negativo")
```

- A) Imprime grande. [S2]
- B) Imprime negativo. [S2]
- **C) No imprime nada.** *(Correcta — la condición exterior es verdadera pero la interior es falsa; no hay else para el if interno.)*
- D) Error de indentación. [S2]

S6. ¿Qué ocurre con este ciclo?

```
n = 0
while n < 5:
    print(n)
```

- A) Imprime 0, 1, 2, 3, 4 y termina. [S4]
- B) Imprime 0 y termina porque n no cambia. [S4]
- **C) Ciclo infinito, imprime 0 repetidamente.** *(Correcta — n nunca se actualiza, por lo que la condición nunca se vuelve falsa.)*
- D) Imprime 0 una sola vez y se detiene, porque while ejecuta el bloque una única vez. [S3]

S7. ¿Qué imprime este código?

```
i = 0
while i < 3:
    print(i)
    if i == 1:
        print("uno")
    i += 1
```

- A) Imprime 0, uno, 1, 2 y termina. [S1]
- **B) Imprime 0, 1, uno, 2 y termina.** *(Correcta — en la iteración donde `i == 1`, primero se imprime `i` y luego "uno"; el ciclo continúa.)*
- C) Imprime 0, 1, uno y termina ahí porque el `if` detiene el ciclo. [S4]
- D) Imprime uno indefinidamente después de la primera vez que `i == 1`. [S3]

S8. ¿Cuál es la diferencia semántica entre `if` y `while`?

- A) No hay diferencia, son intercambiables. [S3]
- B) `while` evalúa la condición al final del bloque; `if` al inicio. [S3, S4]
- **C) `if` evalúa la condición una vez; `while` la evalúa repetidamente al inicio de cada iteración.** *(Correcta — tanto el `if` como el `while` evalúan su condición al inicio, antes de ejecutar cualquier código. La única diferencia es que el `if` lo hace una sola vez y el `while` lo repite en bucle mientras la condición sea verdadera.)*
- D) `if` puede tener `else`; `while` no. [S2]

S9. ¿Qué imprime este código?

```
p = 0
while p < 4:
    p += 2
    print(p)
```

- A) Imprime 0, 2, 4. [S1]
- **B) Imprime 2, 4.** *(Correcta — el incremento ocurre antes del `print`; cuando `p` llega a 4 la condición `p < 4` ya es falsa y el ciclo termina.)*
- C) Imprime 2 y termina porque la condición se cumple. [S4]
- D) Ciclo infinito. [S4]

S10. ¿Qué produce este código?

```
a = 10
b = 0
if a > 5:
    b = 1
if a > 8:
    b = 2
print(b)
```

- A) 1 [S2]
- **B) 2** *(Correcta — ambos `if` se evalúan independientemente; el segundo sobrescribe `b` con 2.)*
- C) 0 [S1]
- D) 0, porque el segundo `if` anula el efecto del primero al evaluarse sobre el mismo valor. [S2]

S11. ¿Qué imprime este código?


```
i = 0
while i < 5:
    if i == 3:
        print("tres")
    i += 1
```

- A) Imprime tres 5 veces. [S3]
- **B) Imprime tres una vez.** *(Correcta — el if solo es verdadero cuando i == 3, lo que ocurre una sola vez en las 5 iteraciones.)*
- C) No imprime nada porque el while y el if se cancelan mutuamente. [S2]
- D) Imprime tres y luego termina el ciclo inmediatamente. [S4]

S12. ¿Qué produce este código?

```
x = 3
if x > 0:
    y = 10
else:
    y = 20
print(y)
```

- A) 20 [S1]
- B) 10 y luego 20. [S2]
- **C) 10** *(Correcta — x > 0 es verdadero, por lo que se ejecuta solo la rama if y y queda en 10.)*
- D) 0 [S1]

S13. ¿Cuántas veces se imprime hola?

```
i = 1
while i <= 3:
    print("hola")
    i += 1
```

- A) 4 veces. [S4]
- **B) 3 veces.** *(Correcta — i toma los valores 1, 2 y 3; cuando i llega a 4 la condición i <= 3 es falsa y el ciclo termina.)*
- C) Solo 1 vez, porque el while ejecuta el bloque una única vez. [S3]
- D) 2 veces, porque la condición deja de cumplirse cuando i == 3 a mitad de iteración. [S4]

S14. ¿Qué imprime este código?

```
x = 0
while x < 3:
    x += 1
    if x == 2:
        print("dos")
print("listo")
```

- A) Imprime dos y termina ahí, sin llegar a listo. [S4]
- **B) Imprime dos una vez y luego listo.** *(Correcta — el if solo se cumple cuando x == 2; el ciclo continúa y print("listo") está fuera del while.)*
- C) Imprime dos tres veces y luego listo. [S3]
- D) Solo imprime listo. [S4]

S15. ¿Qué imprime el siguiente código?

```
activo = True
if activo:
    print("encendido")
```

- A) No imprime nada: `if` solo ejecuta su bloque una vez y como no hay `while`, la condición no se reitera. [S3]
- B) Imprime `encendido` indefinidamente. [S3]
- **C) Imprime `encendido` una vez y termina.** *(Correcta — `if` evalúa la condición una sola vez; al ser `True`, ejecuta el bloque una vez y sigue.)*
- D) Imprime `encendido` y `True` porque el bloque y la condición se ejecutan juntos. [S2]

4.4. Banco F — Errores de Funciones y Parámetros

F1. ¿Qué imprime el siguiente código?

```
def doble(n):
    print(n * 2)
resultado = doble(5)
print(resultado)
```

- **A) Imprime 10 y luego `None`.** *(Correcta — `print` dentro produce el 10 como efecto de salida; la función no tiene `return`, por lo que retorna `None`.)*
- B) Imprime 10 dos veces. [F1]
- C) Imprime solo 10. [F1]
- D) Error por asignar el resultado de una función sin `return`. [F1]

F2. ¿Qué imprime este código?

```
c = 10
def funcion():
    c = 99
funcion()
print(c)
```

- A) 99 [F2]
- **B) 10** *(Correcta — `c = 99` dentro de la función crea una variable local; la `c` global no se modifica.)*
- C) Error: `c` fue redefinida. [F2]
- D) `None`, porque la función sobrescribió la variable global `c` con el valor que retorna implícitamente. [F2]

F3. ¿Qué imprime este código?

```
def calcular(numero):
    return numero * 3
valor = 4
print(calcular(valor))
```

- A) Error: la variable debe llamarse `numero`. [F3]
- **B) 12** *(Correcta — el argumento `valor` (4) se pasa por posición al parámetro `numero`; el nombre de la variable exterior no importa.)*
- C) 4 [F2]
- D) `None` [F1]

F4. ¿Qué retorna la siguiente función al llamarla con `sumar(2, 3)`?

```
def sumar(a, b):  
    resultado = a + b
```

- A) 5 [F1]
- B) `None` (Correcta — sin sentencia `return`, Python retorna `None` implícitamente.)
- C) Error: falta `return`. [F1]
- D) La variable `resultado`. [F2]

F5. ¿Qué imprime este código?

```
def incrementar(x):  
    x += 1  
n = 5  
incrementar(n)  
print(n)
```

- A) 6 [F2]
- B) **5** (Correcta — los enteros son inmutables; `x` es una copia local del valor, modificarla no afecta a `n`.)
- C) `None`, porque la función no retorna nada y eso reemplaza el valor de `n`. [F2]
- D) Error: para que la función afecte a `n`, el parámetro debe llamarse `n`, no `x`. [F3]

F6. ¿Qué imprime el siguiente código?

```
def triple(z):  
    return z * 3  
triple(4)
```

- A) 12 [F1]
- B) **No imprime nada.** (Correcta — el valor se retorna pero no se almacena ni se pasa a `print`, por lo que no hay salida visible.)
- C) Error: el valor retornado no fue capturado. [F1]
- D) El valor 12 queda almacenado en la variable `z` del ámbito global, disponible para usarse después de la llamada. [F2]

F7. ¿Qué imprime este código?

```
total = 0  
def agregar(n):  
    total = total + n  
agregar(5)  
agregar(3)  
print(total)
```

- A) 8 [F2]
- B) 5 [F2]
- C) 0 [F2]
- D) `UnboundLocalError`. (Correcta — Python detecta que `total` se asigna dentro de `agregar`, por lo que la trata como variable local; al intentar leerla antes de asignarla en la misma función, lanza `UnboundLocalError`.)

F8. ¿Qué imprime este código?

```
def suma(a, b):
    return a + b
x = 3
y = 5
print(suma(y, x))
```

- A) Error: los argumentos deben llamarse **a** y **b** para que la función los acepte. [F3]
- **B) 8** (Correcta — los argumentos se pasan por posición: **a** recibe el valor de **y** (5) y **b** recibe el valor de **x** (3); el nombre de la variable no importa.)
- C) Error: **x** e **y** deben llamarse **a** y **b** para que el orden importe. [F3]
- D) 8, pero solo porque **y** se llama igual que el segundo parámetro en otro contexto. [F3]

F9. ¿Qué imprime este código?

```
def mostrar(msg):
    print(msg)
    return msg
r = mostrar("hola")
print(r)
```

- A) Imprime **hola** una sola vez. [F1]
- **B) Imprime hola dos veces.** (Correcta — el **print** interno imprime una vez; **r** recibe el valor retornado y **print(r)** lo imprime nuevamente.)
- C) Imprime **hola** y luego **None**. [F1]
- D) Si la variable **r** se llamara **msg** (igual que el parámetro), recibiría el valor retornado; como no se llama así, **r** queda en **None**. [F3]

F10. ¿Qué valor retorna f(3) dado el siguiente código?

```
def f(x):
    y = x + 1
    return y
    y = y * 2
```

- A) 8, porque la línea **y = y * 2** también se ejecuta antes de retornar. [S1]
- **B) 4** (Correcta — **return y** se ejecuta con **y = 4**; el código después del **return** nunca se alcanza.)
- C) Error: hay código después del **return**. [S1]
- D) La variable local **y** modifica automáticamente cualquier variable global llamada **y**, por lo que el valor retornado sería el de la variable global. [F2]

F11. ¿Qué ocurre si defino def f(numero): y la llamo con f(dato) siendo que la variable dato igual a 7?

- A) Error: **dato** debe llamarse **numero**. [F3]
- **B) La función recibe el valor 7 en el parámetro numero.** (Correcta — el paso de parámetros es por posición; el nombre de la variable en el llamador es irrelevante.)
- C) La función recibe el nombre **dato** como string. [F3]
- D) **numero** queda igual a **dato** y ambas valen 7 en el mismo ámbito. [F2]

F12. ¿Qué imprime el siguiente código?

```
def cuadrado(n):
    return n ** 2
print(cuadrado(4) + 1)
```

- A) `None + 1` (error) [F1]
- B) Solo funcionaría si la variable pasada se llamara `n`; al pasar el literal `4`, la función no puede identificar el parámetro. [F3]
- **C) 17** (Correcta — *cuadrado(4) retorna 16; ese valor se usa directamente en la expresión $16 + 1 = 17$.*)
- D) Error: no se puede operar directamente el retorno de una función. [F1]

F13. ¿Qué pasa si llamo `f(10)` con la función `def f(a, b):`?

- A) `a = 10`, `b = 0` por defecto. [F3]
- B) `a = 10`, `b = None` por defecto. [F3]
- **C) Error: faltan argumentos.** (Correcta — *la función requiere dos argumentos posicionales y solo se pasó uno.*)
- D) `a = 10`, `b` queda indefinida internamente. [F3]

F14. ¿Qué imprime este código?

```
x = 5
def f():
    x = 20
    print(x)
f()
print(x)
```

- A) 20 y luego 20. [F2]
- **B) 20 y luego 5.** (Correcta — *dentro de la función, `x = 20` crea una variable local; la `x` global permanece en 5.*)
- C) Error: la función no puede leer ni modificar la variable `x` del exterior sin declararla con `global`. [F2]
- D) 5 y luego 20. [F2]

F15. ¿Qué produce este código?

```
def f(a, b, c):
    return a + b + c
print(f(1, 2, 3))
```

- A) Error porque hay tres parámetros. [F3]
- **B) 6** (Correcta — *los tres argumentos se pasan por posición y se suman correctamente.*)
- C) `None` [F1]
- D) Error: los parámetros deben llamarse igual que los argumentos. [F3]

4.5. Banco R — Errores de Referencias y Listas

R1. ¿Cómo se accede al primer elemento de `lista = [10, 20, 30]`?

- A) `lista[1]` [R1]
- **B) `lista[0]`** (Correcta — *los índices en Python comienzan en 0; el primer elemento está en la posición 0.*)
- C) `lista[2]` [R1]
- D) `lista[3]` [R1]

R2. ¿Qué imprime este código?

```

lista_1 = [1, 2, 3]
lista_2 = lista_1
lista_2[0] = 99
print(lista_1[0])

```

- A) 1 [R2]
- **B) 99** *(Correcta — lista_2 = lista_1 no crea una copia; ambas variables apuntan al mismo objeto en memoria.)*
- C) 1, porque lista_2 = lista_1 crea una copia independiente y modificar lista_2 no afecta a lista_1. [R2]
- D) [99, 2, 3], porque lista_2 es una copia y al modificarla Python actualiza ambas listas por separado. [R2]

R3. ¿Qué imprime este código?

```

lista = [10, 20, 30, 40]
print(lista[3])

```

- A) 30 [R1]
- **B) 40** *(Correcta — el índice 3 corresponde al cuarto elemento (posición 3 en base-0).)*
- C) Error de índice [R1]
- D) 10 [R1]

R4. ¿Cómo se obtiene una copia independiente de lista = [1, 2, 3]?

- A) copia = lista [R2]
- **B) copia = lista[:]** *(Correcta — el operador rebanador [:] sin índices crea una copia superficial independiente; modificar una no afecta a la otra.)*
- C) copia = lista[0] [R1]
- D) copia = lista[-1] [R1]

R5. ¿Qué imprime este código?

```

lista = [5, 10, 15]
copia = lista
copia.append(20)
print(len(lista))

```

- A) 3 [R2]
- **B) 4** *(Correcta — copia y lista apuntan al mismo objeto; .append(20) modifica ese objeto y lista refleja el cambio.)*
- C) 3, porque copia = lista produce una copia independiente; el append sobre copia no afecta a lista. [R2]
- D) Error de índice [R1]

R6. ¿Qué imprime print(lista[len(lista) - 1]) si lista = [10, 20, 30]?

- A) 10 [R1]
- B) Error de índice [R1]
- **C) 30** *(Correcta — len(lista) - 1 = 2 apunta al último elemento.)*
- D) Si antes se ejecutó otra = lista y se agregó un elemento a otra, el valor de len(lista) NO cambiaría porque otra es una copia independiente. [R2]

R7. ¿Qué ocurre si se ejecuta lista = [1, 2, 3]; print(lista[3])?

- A) Imprime 3. [R1]

- B) Imprime None. [R1]
- C) **IndexError.** *(Correcta — el índice 3 no existe en una lista de 3 elementos (índices 0, 1, 2).)*
- D) Imprime 1. [R1]

R8. ¿Qué imprime este código?

```
a = [1, 2, 3]
b = a[:]
b[0] = 99
print(a[0])
```

- A) 99 [R2]
- B) **1** *(Correcta — a[:] crea una copia independiente; modificar b no afecta a a.)*
- C) 99, porque a[:] solo copia la referencia, no los datos; a y b siguen apuntando al mismo objeto. [R2]
- D) None, porque después del [:] la lista original queda vacía. [R2]

R9. ¿Qué imprime este código?

```
lista = ["a", "b", "c", "d"]
print(lista[1])
```

- A) "a" [R1]
- B) **"b"** *(Correcta — el índice 1 apunta al segundo elemento en base-0.)*
- C) "c" [R1]
- D) Modificar copia[1] no cambiaría lo que imprime esta línea porque copia = lista crea una lista independiente. [R2]

R10. ¿Qué imprime este código?

```
x = [10, 20, 30]
y = x
x = [99]
print(y)
```

- A) [99] [R2]
- B) [10, 20, 30] *(Correcta — x = [99] reasigna x a un nuevo objeto; y sigue apuntando al original.)*
- C) [99, 20, 30] [R2]
- D) [99], porque y = x creó una copia independiente y x = [99] actualizó esa copia, afectando a ambas. [R2]

R11. ¿Cuál es el índice del último elemento de lista = [5, 10, 15, 20]?

- A) 4 [R1]
- B) **3** *(Correcta — una lista de 4 elementos tiene índices 0, 1, 2, 3; el último es 3.)*
- C) 5 [R1]
- D) 2, porque copia = lista crea una lista independiente, así que al agregar elementos a una no cambia el índice máximo de la otra. [R2]

R12. ¿Qué imprime este código?

```
lista = [1, 2, 3]
otra = lista
lista.append(4)
print(otra)
```

- A) [1, 2, 3] [R2]
- B) [1, 2, 3, 4] (Correcta — *otra* y *lista* apuntan al mismo objeto; `.append(4)` modifica ese único objeto.)
- C) [1, 2, 3], porque `otra = lista` crea una copia independiente y el `append` sobre `lista` no la afecta. [R2]
- D) [4], porque `otra` solo recibe los cambios nuevos realizados sobre `lista` después de la asignación. [R2]

R13. ¿Qué imprime este código?

```
lista = [10, 20, 30, 40, 50]
print(lista[0] + lista[4])
```

- A) Error de índice en `lista[4]`. [R1]
- B) `lista[0] + lista[4]` [R1]
- C) **60** (Correcta — *lista[0] = 10* y *lista[4] = 50*; su suma es 60.)
- D) Si `otra = lista` y se modifica `otra[0] = 99`, `lista[0]` seguiría siendo 10, porque `otra` es una copia independiente y los cambios en una lista no afectan a la otra. [R2]

R14. ¿Qué imprime este código?

```
a = [1, 2]
b = a
b = b + [3]
print(a)
```

- A) [1, 2, 3] [R2]
- B) [1, 2] (Correcta — *b + [3]* crea un nuevo objeto y reasigna *b*; *a* sigue apuntando al original.)
- C) [1, 2, 3], porque como *b* apuntaba al mismo objeto que *a*, cualquier cambio en *b* modifica también *a*. [R2]
- D) Error de índice, porque al concatenar `b + [3]` se intenta acceder a una posición que no existe. [R1]

R15. ¿Qué imprime este código?

```
nums = [100, 200, 300]
print(nums[-1])
```

- A) 100 [R1]
- B) Error de índice [R1]
- C) **300** (Correcta — *el índice -1 en Python accede al último elemento de la lista.*)
- D) Si se ejecuta `copia = nums` y luego `copia[-1] = 999`, `nums[-1]` seguiría imprimiendo 300 porque `copia = nums` crea una copia independiente y los cambios en una no afectan a la otra. [R2]

4.6. Banco D — Errores de Diccionarios

D1. Dado el siguiente diccionario, ¿cómo se imprime el precio del artículo "leche"?

```
precios = {"pan": 500, "leche": 800, "jugo": 1200}
```

- A) `print(precios[1])` [D2]
- B) `print(precios["leche"])` (Correcta — los diccionarios se acceden directamente por su clave; no existen índices posicionales.)
- C) `for k in precios: if k == "leche": print(precios[k])` [D1]
- D) `print(precios[2])` [D2]

D2. ¿Qué ocurre al ejecutar el siguiente código?

```
inventario = {"manzana": 10, "pera": 5}  
print(inventario["naranja"])
```

- A) Imprime `None`. [D3]
- B) Imprime `0`. [D3]
- C) **KeyError.** (Correcta — acceder con `[]` a una clave inexistente siempre lanza **KeyError** y detiene el programa.)
- D) No imprime nada y el programa continúa sin error. [D3]

D3. ¿Qué imprime el siguiente código?

```
calificaciones = {"Carolina": 6, "Luis": 5, "Eva": 7}  
for x in calificaciones:  
    print(x)
```

- A) Imprime 6, 5, 7, en algún orden. [D4]
- B) **Imprime Carolina, Luis, Eva, en algún orden.** (Correcta — iterar un diccionario con `for` entrega sus claves, no sus valores.)
- C) Imprime Carolina 6, Luis 5, Eva 7. [D4], en algún orden.
- D) Imprime sólo la primera clave (Carolina), porque para obtener todas hay que recorrer el diccionario con un `for` que compare clave por clave con `==`. [D1]

D4. ¿Cómo se agrega la entrada "ciudad": "Santiago" al diccionario persona?

- A) `persona.add("ciudad", "Santiago")` [D1]
- B) `persona.append("ciudad", "Santiago")` [D1]
- C) `persona["ciudad"] = "Santiago"` (Correcta — la forma estándar de agregar o actualizar una entrada es asignar directamente a la clave.)
- D) `persona[len(persona)] = "Santiago"` [D2]

D5. ¿Qué imprime este código?

```
votos = {"candidato_a": 150, "candidato_b": 200}  
total = 0  
for clave in votos:  
    total += clave  
print(total)
```

- A) 350 [D4]
- B) **TypeError.** (Correcta — `clave` toma los strings `"candidato_a"` y `"candidato_b"`; sumarlos a un `int` lanza **TypeError**.)
- C) `"candidato_acandidato_b"` [D4]

- D) 350, porque aunque `for clave in votos` entrega las claves, Python las convierte automáticamente al tipo del acumulador antes de sumar. [D4]

D6. ¿Qué imprime este código?

```
países = {"primero": "Chile", "segundo": "Peru", "tercero": "Bolivia"}
print(países[0])
```

- A) "Chile" [D2]
- B) "primero" [D2]
- C) **KeyError.** (Correcta — el diccionario no tiene la clave 0; los diccionarios no se indexan por posición.)
- D) None [D3]

D7. ¿Cuál es la forma correcta de verificar si la clave "stock" existe en `d` antes de acceder a su valor?

- A) `if d["stock"] != None:` [D3]
- B) `if "stock" in d:` (Correcta — el operador *in* sobre un diccionario comprueba la existencia de la clave sin lanzar error.)
- C) Recorrer el diccionario con `for k in d` y comparar cada `k`. [D1]
- D) `if d[0] == "stock":` [D2]

D8. ¿Qué imprime el siguiente código?

```
temperaturas = {"lunes": 18, "martes": 22}
for dia in temperaturas:
    print(temperaturas[dia])
```

- A) Imprime `lunes`, `martes`, en algún orden.. [D4]
- B) **Imprime 18, 22, en algún orden.** (Correcta — *dia* toma las claves; `temperaturas[dia]` accede al valor asociado a cada clave.)
- C) Error: sólo se pueden usar índices numéricos entre [], no variables de tipo string como `dia`. [D2]
- D) **KeyError**, porque `dia` toma los valores numéricos 18 y 22, y esos números no son claves del diccionario. [D4]

D9. ¿Qué imprime este código?

```
notas = {"matematica": 6, "historia": 5, "ciencias": 7}
print(notas["historia"])
```

- A) 6 [D2]
- B) **5** (Correcta — `notas["historia"]` accede directamente al valor asociado a la clave "historia".)
- C) "historia", porque acceder a un diccionario con una clave retorna la propia clave consultada. [D3]
- D) Error: no se puede acceder con una clave de tipo string. [D2]

D10. ¿Qué imprime este código?

```
stock = {"manzana": 10}
stock["pera"] = 5
print(len(stock))
```

- A) 1, porque una clave nueva solo se agrega si ya existía previamente en el diccionario. [D3]

- **B) 2** (Correcta — la asignación `stock["pera"] = 5` agrega un nuevo par clave-valor; el diccionario ahora tiene 2 entradas.)
- C) Error: `len` solo cuenta elementos accesibles por índice numérico y los diccionarios no tienen índices posicionales. [D2]
- D) 5, porque `len` retorna el valor de la última clave agregada cuando esa clave no existía antes. [D3]

D11. ¿Qué ocurre al ejecutar el siguiente código?

```
d = {"a": 1, "b": 2}
valor = d["c"]
print(valor)
```

- A) Imprime `None`. [D3]
- B) Imprime 0. [D3]
- **C) `KeyError` en la línea `valor = d["c"]`.** (Correcta — el operador `[]` lanza `KeyError` y detiene el programa antes de llegar al `print`.)
- D) Imprime `"c"`, porque cuando la clave no existe el diccionario retorna la propia clave consultada como valor por defecto. [D3]

D12. ¿Qué imprime este código?

```
frutas = {"manzana": 3, "pera": 5, "uva": 2}
for fruta in frutas:
    if fruta > 3:
        print(fruta)
```

- A) Imprime `pera`. [D4]
- B) Imprime 5. [D4]
- **C) `TypeError`.** (Correcta — `fruta` toma los strings `"manzana"`, `"pera"`, `"uva"`; comparar un string con 3 lanza `TypeError`.)
- D) No imprime nada, porque `fruta` toma los strings y al compararlos con 3 el resultado siempre es `False` sin lanzar error. [D4]

D13. ¿Qué pasa al ejecutar `ventas["enero"] += 100` si la clave `"enero"` no existe en `ventas`?

- A) La clave se crea con valor 100. [D3]
- B) La clave se crea con valor `None`. [D3]
- **C) `KeyError`.** (Correcta — `+=` necesita leer `ventas["enero"]` primero; si no existe, lanza `KeyError`.)
- D) La clave se ignora y el diccionario no cambia. [D3]

D14. ¿Qué imprime este código?

```
d = {"a": 10, "b": 20, "c": 30}
total = 0
for k in d:
    total = total + d[k]
print(total)
```

- **A) 60** (Correcta — `k` toma cada clave; `d[k]` obtiene el valor numérico y se acumula correctamente: $10 + 20 + 30 = 60$.)
- B) `TypeError`. [D4]
- C) 0 [D1]
- D) `abc` [D4]

D15. Un estudiante quiere recorrer el diccionario edades e imprimir la edad de cada persona. Analiza el siguiente código:

```
edades = {"Ana": 22, "Pedro": 19, "Maria": 25}
for persona in edades:
    print(persona + 1)
```

¿Qué produce este código?

- A) Imprime 23, 20, 26. [D4]
- B) Imprime Ana1, Pedro1, Maria1. [D4]
- C) TypeError. (*Correcta — **persona** toma los strings "Ana", "Pedro", "Maria"; sumar 1 a un string lanza TypeError.*)
- D) Imprime Ana, Pedro, Maria sin el + 1. [D1]